

Bloque 01 — Fundamentos de datos

Propósito

Antes de programar, Leo necesita aprender a mirar un conjunto de datos como una representación limitada de una operación real. Usaremos durante todo el bloque el caso de **Mercado Faro**, un marketplace con una web y una app: usuarios crean cuentas, hacen pedidos compuestos por líneas de producto y generan eventos de uso.

Resultado de salida

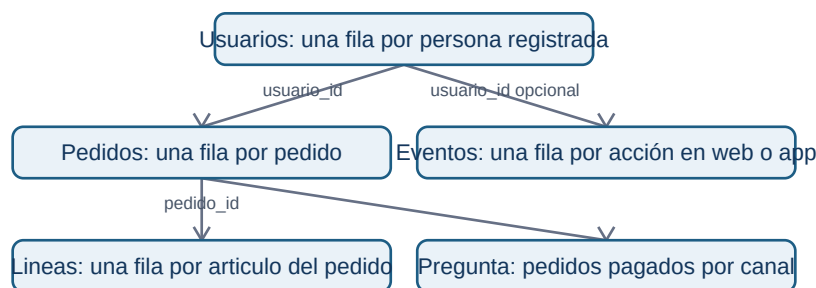
Al acabar podrás explicar qué representa cada archivo y cada fila; elegir el grano adecuado; relacionar usuarios, pedidos, líneas y eventos sin multiplicar resultados; leer CSV y JSON con precaución; y documentar controles de calidad, privacidad y trazabilidad antes de recomendar una decisión.

Prerrequisitos

Ninguno. Los términos archivo, tabla, CSV, JSON, clave y relación se construyen desde cero.

Caso continuo

La dirección pregunta: «¿cuántos pedidos pagados tuvimos ayer y qué canal conviene mejorar?». Esta pregunta parece simple, pero obliga a distinguir personas de pedidos, pedidos de líneas de producto y acciones dentro de una app. Cada lección añade una pieza al mismo mapa.



El diagrama no dice que todas las tablas puedan sumarse entre sí: muestra qué identificador permite conectar cada hecho sin cambiar su significado.

Lecciones

1. Archivo, tabla, observación y grano.
2. Entidades, eventos, claves, relaciones y joins.
3. CSV, JSON y conversión a tablas analizables.
4. Contrato, calidad, privacidad y trazabilidad.

Práctica y laboratorio

- Resuelve la auditoría del marketplace y consulta después la solución razonada.
- Ejecuta `notebooks/practicas/01-fundamentos-marketplace.py`. No requiere instalar librerías: lee los archivos de ejemplo, verifica reglas y demuestra cómo un join mal planteado cambia una cifra.
- Los archivos mínimos del caso están en `datasets/temario-01/`.

Criterio de dominio

No sigas al bloque de Python hasta poder completar, para cada tabla: «cada fila representa...», «su identificador es...», «esta métrica se calcula contando/sumando...» y «estos datos no permiten concluir...».

01.1 Archivo, tabla, observación y grano

Objetivos y prerequisites

Al terminar podrás distinguir archivo, tabla, fila, columna y celda; escribir el **grano** de una fuente; y explicar por qué contar filas no siempre equivale a contar personas. No se presupone vocabulario técnico.

Una pregunta cotidiana antes de la jerga

Mercado Faro quiere saber cuántos pedidos pagados recibió ayer. El sistema no guarda «la respuesta»; guarda huellas de cosas que ocurrieron: una persona se registró, pulsó un botón, creó un pedido o añadió dos artículos. Un **dato** es una representación registrada de una parte de la realidad, no la realidad completa.

Un **archivo** es una unidad con nombre y contenido que se puede guardar, copiar y abrir, como una foto o una nota. Si el contenido sigue filas y columnas, puede representar una **tabla**. Una tabla organiza observaciones comparables: una **fila** contiene un caso; una **columna** describe la misma propiedad

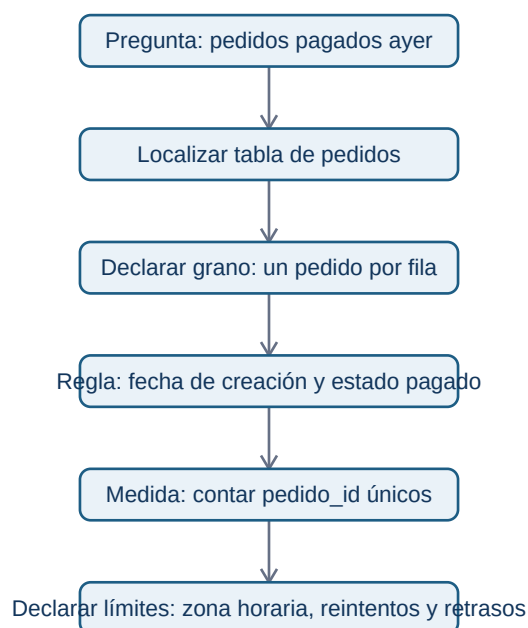
para todos los casos; una **celda** es el cruce de ambas.

Aquí cada fila es una observación de **un pedido**, no de un usuario. U-10 aparece dos veces porque hizo dos pedidos. El nombre técnico de esta precisión es el **grano**: la unidad que representa exactamente una fila.

El grano determina qué cálculo responde a qué pregunta

Antes de abrir una herramienta completa esta frase: «cada fila de esta tabla representa ____». Si la respuesta es «un pedido», contar tres filas responde «tres pedidos»; no «tres clientes». Para clientes únicos se cuentan valores distintos de `usuario_id`; para facturación pagada se suman `total_eur` solo donde `estado = pagado`.

¿Qué camino convierte la pregunta en una cifra defendible?



La cifra no es solo un COUNT: depende de la definición de «ayer», del estado que cuenta como pago y de que `pedido_id` no esté duplicado.

Cuatro granos que no se pueden intercambiar

Una **entidad** es algo relativamente estable que queremos identificar, como usuario o producto. Un **evento** es algo que ocurre en un momento, como `checkout_iniciado`. Un pedido es una

transacción: registra un intercambio u operación de negocio. En la siguiente lección se afina esta distinción.

Ejemplo trabajado: un total que parece correcto y no lo es

El pedido P-100 tiene dos líneas: camiseta (20 €) y envío (2 €); P-102 tiene tres líneas por 65 €. Si unimos pedidos con líneas y después contamos filas, veremos cinco filas y podríamos decir «cinco pedidos». Es falso: hay dos pedidos. La suma de `pedidos.total_eur` tras ese join también se repetirá una vez por línea, inflando los ingresos.

El problema no es el software: es haber olvidado el grano al cambiar de tabla. Conserva siempre una nota junto al análisis: fuente, grano, filtro, periodo y unidad.

Límites y error frecuente

No asumas que una tabla contiene todo. Los eventos pueden faltar si una persona usa bloqueador; un pedido puede estar pendiente de pago; la hora puede venir en UTC mientras la dirección habla de Madrid. Una tabla es evidencia parcial y debe leerse junto con su cobertura y reglas.

Resumen y comprobación

- Archivo: contenido guardado con un nombre y un formato.
 - Tabla: organización en filas y columnas.
 - Grano: lo que representa una fila; dicta qué se puede contar o sumar.
1. Escribe el grano de una tabla de sesiones y otro de una tabla de usuarios.
 2. ¿Por qué tres filas con el mismo `usuario_id` no son necesariamente un error?
 3. Para «productos vendidos», ¿usarías pedidos o líneas de pedido? Justifica la elección.

Aplica estas ideas en [la práctica del marketplace](#).

01.2 Entidades, eventos, claves, relaciones y joins

Objetivos y prerrequisitos

Aprenderás a diseñar y leer relaciones entre tablas, distinguir clave primaria y foránea, comprobar cardinalidades 1:1, 1:N y N:M, y detectar un join que multiplica filas. Parte de la lección 01: ya sabes que cada tabla tiene grano.

Del mundo real a varias tablas pequeñas

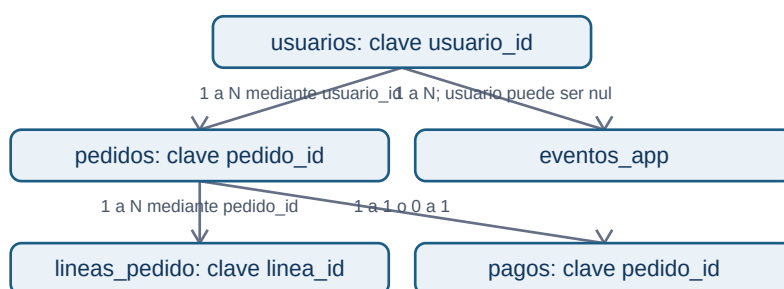
Guardar nombre, dirección y producto repetidos en cada pedido vuelve los datos difíciles de corregir y de analizar. Separamos la información según lo que representa: `usuarios` para personas registradas, `pedidos` para transacciones y `lineas_pedido` para artículos del pedido. Una **dimensión** suele describir una entidad (por ejemplo, producto o usuario); una tabla de hechos registra eventos o transacciones medibles.

También existe un **snapshot**: una fotografía de estado en un instante. Una tabla con una fila por producto y día que guarda su stock al cierre no es un evento de cambio de stock: es el estado observado a esa fecha. Confundir ambos altera tendencias y acumulados.

Claves: etiquetas para reconocer y conectar

Una **clave primaria** identifica de manera única una fila dentro de su tabla: `usuarios.usuario_id` o `pedidos.pedido_id`. Una **clave foránea** guarda la referencia a otra tabla: `pedidos.usuario_id` apunta al usuario que hizo el pedido. Que sea numérica o de texto no la convierte en medida: no tiene sentido promediar identificadores.

¿Cómo se conectan las piezas del caso sin inventar relaciones?



Un usuario puede tener muchos pedidos (1:N); cada pedido pertenece a un usuario si el negocio lo exige. Un pago puede ser 0:1 con pedido si hay pedidos iniciados pero no cobrados. Las relaciones describen una regla de negocio, no una forma de dibujo.

Cardinalidad y tabla puente

- **1:1**: una fila A se asocia como máximo a una fila B; por ejemplo, pedido y comprobante de pago final si el sistema lo garantiza.
- **1:N**: un usuario puede tener N pedidos; cada pedido tiene un usuario.
- **N:M**: un pedido puede incluir N productos y un producto aparece en M pedidos. No se conecta directamente: `lineas_pedido` actúa como **tabla puente** con `pedido_id`, `producto_id`, cantidad y precio.

La tabla puente no es una molestia técnica: conserva el grano «un artículo de un pedido» y permite responder unidades por producto sin repetir atributos del pedido.

Join: combinar solo con una hipótesis verificable

Un **join** combina filas según una clave. Antes de ejecutarlo escribe: (1) grano de la tabla izquierda, (2) grano de la derecha, (3) cardinalidad esperada, (4) qué métrica se mantendrá. Si pedidos (uno por pedido) se une a `lineas_pedido` (varias por pedido), el resultado tendrá grano de línea, no de pedido.

Ejemplo: P-100 total 42 € y dos líneas. Tras el join aparecerá dos veces con 42 €. Sumar `total_eur` da 84 € para ese pedido: una multiplicación silenciosa. Para facturación, agrega las líneas por pedido primero o calcula la métrica en pedidos antes de unir dimensiones.

Un caso especialmente peligroso es unir dos tablas que tienen varias filas por `usuario_id` (eventos y pedidos). Si U-10 tiene 3 eventos y 2 pedidos, el join produce 6 filas. Esa tabla no representa ni eventos ni pedidos originales.

Diccionario y contrato de datos

El **diccionario de datos** define el significado de cada columna. Un **contrato de datos** además expresa reglas compartidas entre quien produce y quien consume la fuente: esquema, grano, claves, actualización, valores válidos y responsable.

Un contrato evita que «total» cambie de incluir a excluir IVA sin aviso. También permite investigar una incidencia: versión de fuente, momento de carga y responsable dejan **trazabilidad**.

Error frecuente, resumen y comprobación

No des por única una clave por el nombre ni des por válida una relación por tener la misma columna. Comprueba nulos, duplicados y número de filas antes y después de cada join.

1. ¿Cuál es el grano del resultado de unir pedidos con `lineas_pedido`?
2. Da un ejemplo realista de relación N:M distinta de productos y pedidos.
3. ¿Qué regla del contrato impediría contar dos veces el mismo pedido?

Resuelve las preguntas de joins en [la práctica](#).

01.3 CSV, JSON y conversión a tablas analizables

Objetivos y prerequisites

Sabrás leer un CSV y un JSON como archivos de texto, explicar sus diferencias operativas, reconocer separador, codificación y fechas, y convertir un JSON de pedidos en tablas con grano claro. Se parte de archivo y tabla, no de experiencia con programación.

CSV: una tabla escrita línea a línea

Un **CSV** (*comma-separated values*) guarda una tabla como texto. La primera línea suele ser el encabezado; cada línea posterior, una fila. El nombre es histórico: en España es frecuente usar punto y coma para no confundir la coma decimal con el separador.

```
pedido_id;creado_en_utc;total_eur;canal
P-100;2026-07-24T07:14:00Z;42,00;web
P-102;2026-07-24T18:22:00Z;65,00;app
```

Antes de tratarlo como tabla hay que acordar el **dialecto**: separador ;, decimal , , codificación de caracteres (preferiblemente UTF-8), comillas para texto con separadores y formato de fecha. Si una herramienta espera coma como separador, puede leer toda la línea como una sola columna; si interpreta 42,00 en otro contexto, puede dejarlo como texto.

La fecha 2026-07-24T07:14:00Z sigue ISO 8601: Z significa UTC. No la cambies a «hora local» sin declarar zona y regla de conversión.

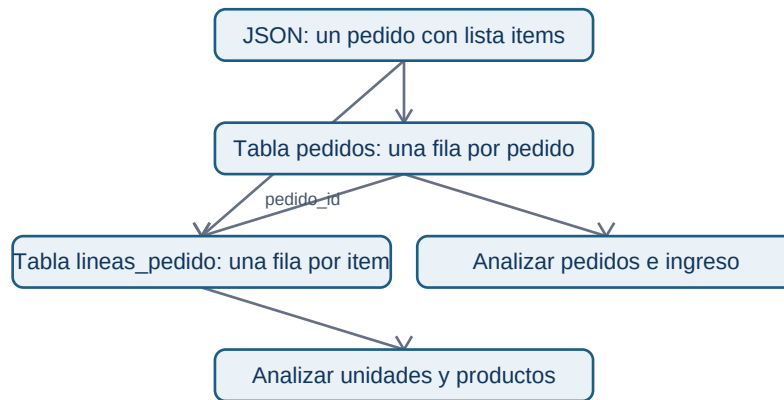
JSON: una ficha con estructura interna

Un archivo **JSON** (*JavaScript Object Notation*) también es texto, pero puede contener objetos y listas. Es habitual al recibir datos de una API: un servicio responde con una ficha de pedido que incluye al usuario y sus artículos.

```
{
  "pedido_id": "P-100",
  "usuario": {"usuario_id": "U-10", "pais": "ES"},
  "items": [
    {"producto_id": "PR-7", "cantidad": 1, "precio_eur": 20.0},
    {"producto_id": "PR-9", "cantidad": 1, "precio_eur": 20.0}
  ],
  "total_eur": 42.0
}
```

}

No hay una única conversión correcta de JSON a CSV. El objeto principal se puede convertir en una fila de pedidos; `usuario.usuario_id` se extrae como una columna; cada elemento de `items` debe crear una fila de `lineas_pedido`. Repetir el pedido en cada línea es válido solo si declaramos que el resultado tiene grano de línea y no reutilizamos `total_eur` como si fuera un importe por línea.



El objetivo de la conversión no es «aplanar todo»: es conservar significado y poder analizar cada pregunta con el grano correspondiente.

Otros medios y elección razonada

Excel es una aplicación y un formato útil para revisión humana y casos pequeños, pero varias ediciones manuales sin historial dificultan reproducir un análisis. Parquet guarda datos por columnas y tipos de forma eficiente; suele usarse con herramientas de datos, no editándose a mano. Una base de datos mantiene datos compartidos con consultas, permisos y reglas; SQL, MongoDB o DynamoDB se verán más adelante con profundidad.

La elección responde a una necesidad: CSV para intercambio de tabla simple; JSON para respuestas estructuradas; Parquet para volúmenes tabulares y procesos analíticos; base de datos para operación concurrente. Ningún formato arregla un grano o una definición defectuosos.

Contraejemplos y comprobación

No abras un CSV «a doble clic» y des por hecho que se interpretó bien. Verifica columnas, filas, tipos, caracteres como ñ y fechas. No conviertas una lista JSON en una sola celda para luego intentar contar productos.

1. ¿Qué separador y decimal usa el CSV del ejemplo?
2. ¿Qué dos tablas crearías a partir del JSON y cuál sería su clave de unión?

3. ¿Por qué total_eur no se debe sumar sin cuidado tras expandir items?

El laboratorio ejecutable muestra una conversión deliberadamente pequeña y auditable.

01.4 Contrato, calidad, privacidad y trazabilidad

Objetivos y prerequisites

Aprenderás a convertir «revisar datos» en reglas observables, clasificar incidencias por severidad, investigar ausencias sin borrarlas por costumbre y limitar el uso de información personal. Requiere comprender grano, claves y formatos de las lecciones anteriores.

Calidad no significa perfección

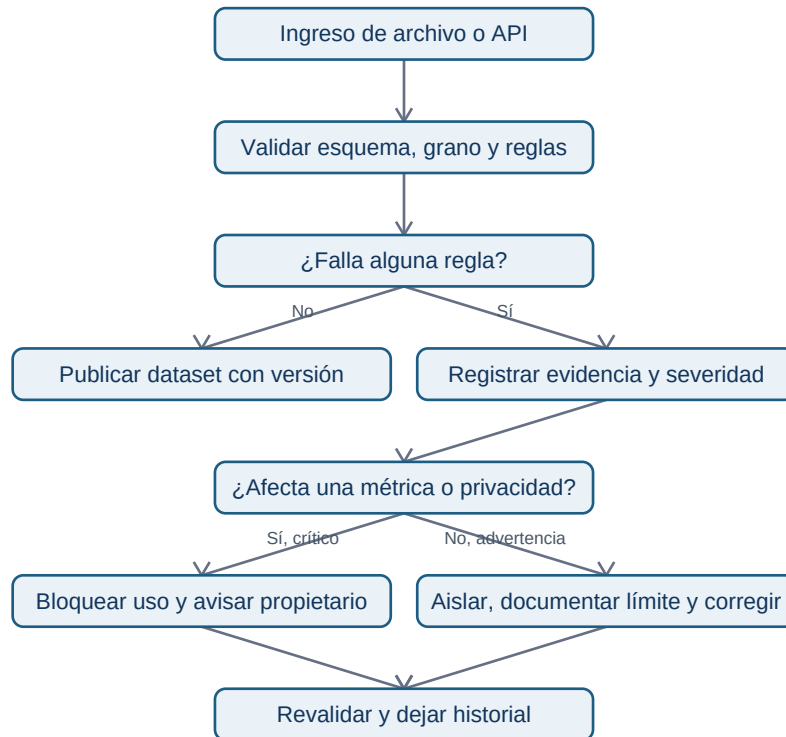
Un conjunto es de calidad suficiente si sirve para una decisión concreta con límites conocidos. Los pedidos de Mercado Faro pueden ser aptos para planificar empaquetado diario y no para estudiar satisfacción, porque no contienen opiniones. Antes de calcular una métrica, documenta qué debe ser cierto.

La calidad necesita responsable y reacción, no solo una lista. Una regla fallida se registra con fecha, fuente, número de filas afectadas, severidad, decisión y seguimiento: eso es **trazabilidad**.

De regla a decisión: severidad y contrato

El contrato de datos de la lección 02 declara esquema, grano, reglas, propietario y frecuencia. Ahora añadimos severidad. Un pedido_id duplicado que infla ingresos es **crítico**: se bloquea el reporte. Un canal nuevo affiliate puede ser una advertencia: se aísla, se consulta a Growth y no se adivina a qué categoría pertenece. Una descripción de producto vacía quizá sea informativa y no impida el cálculo de pedidos.

¿Cómo se gobierna una incidencia sin esconderla bajo una limpieza automática?



El flujo enseña que «limpiar» no equivale a borrar. Primero se preserva evidencia; después se decide una corrección reproducible.

Ausencias, sesgo y cobertura

Un vacío puede significar «no aplica», «no se capturó», «falló el tracking» o «la persona no quiso responder». Si falta sobre todo en usuarios de la app antigua, eliminar esas filas altera la población y puede esconder un fallo técnico. Mide ausencia por fecha, versión, canal y segmento; declara quién queda fuera antes de concluir que un canal rinde peor.

El **sesgo de cobertura** aparece cuando la fuente representa peor a parte de la población. Observar que quienes activaron notificaciones compran más no prueba que activar notificaciones cause compras: pueden ser usuarios ya más interesados. Un analista separa observación, explicación posible y decisión que aún requiere evidencia.

Privacidad: finalidad, minimización y retención

Los datos personales identificables (**PII**, por *personally identifiable information*) son datos que identifican o pueden ayudar a identificar a una persona, como correo, teléfono, dirección o combinaciones poco frecuentes. Para contar pedidos por canal no necesitamos el correo. Aplicamos:

- **Finalidad:** define para qué se usa cada campo antes de recogerlo o consultarlo.
- **Minimización:** usa solo los campos necesarios; sustituye identificadores por un ID interno cuando sea posible.
- **Acceso:** limita quién puede ver PII y no copies datos reales en notebooks, ejercicios o capturas.
- **Retención:** fija cuánto tiempo se conserva y cómo se elimina o anonimiza según la política y normativa aplicables.

Pseudonimizar no vuelve un conjunto automáticamente anónimo: un identificador sustituido aún puede relacionarse con una persona si existe la tabla de correspondencia o combinaciones reidentificables. Para decisiones legales o de tratamiento real, intervienen responsables de privacidad y la normativa vigente; el analista no debe improvisar permisos.

Resumen y comprobación

Una métrica defendible exige grano, contrato y controles. La ausencia es un resultado que se investiga; calidad y privacidad son condiciones del análisis, no una fase administrativa final.

1. Clasifica como crítica, advertencia o informativa una fecha nula en un pedido pagado y justifica.
2. ¿Por qué borrar todos los nulos de país puede sesgar una comparación por canal?
3. Para un dashboard de pedidos por canal, ¿qué PII puedes excluir?

Completa [la auditoría](#) y ejecuta el laboratorio antes de pasar a Python.